

Navegación de costo mínimo sobre terreno real mediante Aprendizaje por Refuerzo Profundo

Andres Flores , Victor Lazo , Rubén López , Juan Quispe 

Unidad de Posgrado - Maestría en Inteligencia Artificial

Facultad de Ingeniería Industrial y de Sistemas

Universidad Nacional de Ingeniería

Lima, Perú

Resumen—En el presente artículo se aborda el problema del aprendizaje de navegación autónoma de costo mínimo sobre topografía real mediante Aprendizaje por Refuerzo Profundo (DRL). A diferencia del enfoque previo basado en Q-learning tabular con condiciones fijas, este trabajo incrementa la complejidad del entorno utilizando un modelo digital de elevación a resolución completa (108×108 celdas de 30 m), donde el agente opera bajo observación parcial y estados iniciales aleatorios en cada episodio. Se aproxima la función de valor de la acción mediante una red Deep Q-Network (DQN) y se realiza una evaluación sistemática comparando métodos basados en valor (DQN y Double DQN) y basados en política (REINFORCE), empleando el algoritmo de Dijkstra como línea base de optimalidad. Los resultados experimentales demuestran la viabilidad de la propuesta: el agente entrenado generaliza con éxito hacia posiciones iniciales no observadas, describiendo trayectorias cercanas al óptimo y mostrando robustez ante transiciones estocásticas. Adicionalmente, se caracterizan y resuelven dos problemas críticos de diseño: el colapso del aprendizaje inducido por el rediseño de recompensa (*reward shaping*) potencial bajo descuento temporal, solucionado mediante un esquema de progreso, y la divergencia asintótica de los valores Q en configuraciones con objetivos aleatorios, fundamentando la adopción de un esquema de meta fija.

Palabras claves—Aprendizaje por Refuerzo Profundo, Deep Q-Network, REINFORCE, navegación, reward shaping, modelo digital de elevación.

Abstract—This paper addresses the problem of learning minimum-cost autonomous navigation over real topography using Deep Reinforcement Learning (DRL). Unlike the previous approach based on tabular Q-learning with fixed conditions, this work increases environment complexity by using a full-resolution digital elevation model (108×108 cells of 30 m), where the agent operates under partial observation and randomized initial states in each episode. The action-value function is approximated using a Deep Q-Network (DQN), and a systematic evaluation is conducted to compare value-based methods (DQN and Double DQN) with policy-based methods (REINFORCE), using Dijkstra's algorithm as the optimality baseline. Experimental results demonstrate the feasibility of the proposed approach: the trained agent successfully generalizes to unobserved starting positions, generating near-optimal trajectories and exhibiting robustness to stochastic transitions. Additionally, we characterize and resolve two critical design issues: learning collapse induced by potential-based reward shaping under temporal discounting,

solved through a progress-based scheme, and the asymptotic divergence of Q -values in configurations with randomized targets, justifying the adoption of a fixed-target scheme.

Keywords—Deep Reinforcement Learning, Deep Q-Network, REINFORCE, navigation, reward shaping, digital elevation model.

I. INTRODUCCIÓN

La navegación y planificación de rutas de costo mínimo sobre superficies topográficas reales representa un problema clásico en optimización. Ante la disponibilidad de mapas completos y entornos deterministas, algoritmos como el de Dijkstra garantizan la obtención del camino óptimo global. No obstante, un agente autónomo real opera bajo observación parcial y debe tomar decisiones locales con información incompleta. En tales escenarios, el Aprendizaje por Refuerzo (Reinforcement Learning, RL) provee un marco formal para el aprendizaje de políticas de navegación mediante interacción directa con el entorno.

En la fase previa del proyecto, el terreno fue modelado como un Proceso de Decisión de Markov (Markov Decision Process, MDP) y resuelto mediante Q-learning tabular en una cuadrícula reducida de 54×54 celdas con origen y destino fijos. Dicho enfoque almacena una matriz de valores y presenta severas limitaciones de escalabilidad y generalización, restringiendo la adaptabilidad del agente a condiciones o puntos de inicio no observados durante el entrenamiento.

En el presente estudio se incrementa la complejidad del escenario y se aplica Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning, DRL). Las principales contribuciones de este trabajo son:

- Un entorno de mayor complejidad caracterizado por una resolución topográfica completa (108×108 celdas), observación parcial (un parche local de 9×9 celdas y el vector relativo hacia la meta) e inicio aleatorio, lo cual demanda capacidades de generalización en el agente y sustenta la transición hacia aproximadores de función neuronales.
- Un análisis comparativo entre métodos basados en valor (Deep Q-Network y Double DQN) y métodos basados en política (REINFORCE) sobre el mismo terreno físico, empleando el algoritmo de Dijkstra como línea base de optimalidad.

Correos electrónicos:

(A. Flores) affloresb@uni.pe

(V. Lazo) vlazop@uni.pe

(R. Lopez) ruben.lopez.m@uni.pe

(J. Quispe) juan.quispe.h@uni.pe

- La caracterización experimental y resolución de dos problemáticas de diseño: la trampa de incentivos inducida por el rediseño de recompensa (*reward shaping*) potencial bajo descuento temporal y la inestabilidad/divergencia de los valores Q en entornos con metas aleatorias.

En cuanto a la metodología, el terreno real se modela como un MDP a partir de un Modelo Digital de Elevación (Digital Elevation Model, DEM), donde el costo de tránsito es proporcional a la pendiente local. El agente recibe una observación en forma de parche local y el vector relativo al objetivo, aproximando la función $Q(s, a)$ mediante un perceptrón multicapa optimizado con DQN, memoria de reproducción de experiencias y red objetivo. El entrenamiento se realiza bajo un esquema de aprendizaje curricular (*curriculum learning*) incrementando progresivamente la distancia del estado inicial. La evaluación de la política codiciosa (*greedy*) se lleva a cabo sobre 20 configuraciones de inicio no observadas durante el entrenamiento, cuantificando la tasa de éxito y la brecha de costo en comparación con la ruta óptima de Dijkstra.

El resto del documento se estructura de la siguiente manera. La Sec. II revisa la literatura científica previa y contextualiza la contribución de este trabajo. La Sec. III describe el fundamento matemático de los algoritmos seleccionados. La Sec. IV formaliza el MDP, el espacio de observaciones y la estructura de la recompensa. La Sec. V detalla los aspectos de implementación, el pseudocódigo y los hiperparámetros. La Sec. VI presenta los resultados cuantitativos y la Sec. VII discute sus implicancias. Finalmente, la Sec. VIII presenta las conclusiones y en la Sec. IX líneas de trabajo futuro.

II. TRABAJOS RELACIONADOS

II-A. Planificación clásica de rutas

La resolución del problema de la ruta de costo mínimo sobre mallas discretas se encuentra ampliamente documentada en la literatura. El algoritmo de Dijkstra [1] proporciona el óptimo global exacto en tiempo polinomial, y sus variantes heurísticas optimizan la eficiencia de búsqueda. Sin embargo, dichos métodos requieren el conocimiento completo del grafo y un entorno estrictamente determinista. En el presente estudio, el algoritmo de Dijkstra se emplea como una línea base de optimalidad para evaluar el desempeño de un agente sujeto a observación parcial y transiciones estocásticas, dos supuestos que la planificación clásica no aborda de manera directa.

II-B. RL tabular sobre grillas

El entorno de tipo cuadrícula (*gridworld*) constituye el banco de pruebas estándar del Aprendizaje por Refuerzo [8], donde el algoritmo Q-learning tabular ofrece convergencia cuando el espacio de estados es acotado. En el trabajo preliminar se implementó dicha aproximación bajo una configuración de malla reducida (54×54 celdas) con origen y destino fijos, requiriendo una tabla de valores de dimensiones manejables. No obstante, al escalar el problema a una cuadrícula completa de 108×108 celdas con puntos de inicio aleatorios, la formulación tabular resulta inviable debido a que el espacio

combinado exigiría un orden de 10^8 entradas. Adicionalmente, esta aproximación carece de la propiedad de transferencia de conocimiento entre estados similares, memorizando trayectorias en lugar de generalizar políticas de navegación.

II-C. Métodos profundos basados en valor

DQN [2] soluciona la limitación de la tabla mediante el uso de una red neuronal aproximadora y estabiliza el proceso de entrenamiento incorporando dos mecanismos clave: la memoria de reproducción de experiencias (*experience replay*) y una red objetivo congelada. Por su parte, Double DQN [3] mitiga el sesgo optimista derivado del operador de maximización en la actualización de Bellman. Liu y Zou [4] evaluaron de forma sistemática el impacto de la memoria de reproducción en entornos de la suite de Atari. En este trabajo se realiza un análisis de ablación análogo sobre un Modelo Digital de Elevación real, operando sobre un espacio de observaciones de baja dimensión (83 características de entrada), un escenario donde la contribución de cada componente para la estabilidad del aprendizaje no resultaba obvia *a priori*.*

II-D. Métodos basados en política

REINFORCE [6] optimiza los parámetros de la política de forma directa empleando el teorema del gradiente de la política, constituyendo el fundamento de la familia de algoritmos actor-crítico [7]. Las evaluaciones comparativas entre métodos basados en valor y basados en política suelen realizarse sobre entornos sintéticos estandarizados (e.g., Atari o MuJoCo). El presente trabajo realiza una comparación directa entre DQN y REINFORCE evaluados sobre el mismo entorno topográfico y bajo las mismas métricas experimentales, con el propósito de caracterizar el comportamiento de la estabilidad y la varianza en una tarea de navegación específica.

II-E. Diseño de la recompensa

Ng, Harada y Russell [5] demostraron que el rediseño de recompensas (*reward shaping*) estructurado bajo una función de diferencia potencial de la forma $F = \gamma \Phi(s') - \Phi(s)$ garantiza la preservación de la política óptima. Si bien esta formulación teórica es correcta y se adopta frecuentemente como directriz de diseño, en este trabajo se reporta un escenario práctico donde esta metodología induce la convergencia hacia políticas subóptimas. Específicamente, bajo factores de descuento $\gamma < 1$ y distancias de tránsito prolongadas, el residuo temporal del descuento recompensa al agente por oscilar repetidamente en regiones lejanas a la meta (Sec. VII). La caracterización y documentación cuantitativa de esta anomalía en problemas de navegación sobre topografía real constituye una de las contribuciones empíricas del presente estudio.

II-F. Políticas condicionadas a la meta

En tareas de aprendizaje con objetivos variables (*goal-conditioned*), los enfoques comunes involucran el condicionamiento de la función de valor respecto al objetivo (UVFA) [9], el reetiquetado de trayectorias (Hindsight Experience Replay, HER) [11] o el muestreo priorizado del búfer (PER)

[10]. El alcance del presente trabajo se limita a caracterizar la problemática que motiva dichos desarrollos, demostrando que la aplicación de DQN estándar en entornos con metas aleatorias induce la divergencia de los valores Q y el colapso de la tasa de éxito, lo cual fundamenta la adopción de una meta fija y establece la base para investigaciones futuras.

II-G. Resumen del aporte

Frente a la literatura revisada, este estudio realiza tres contribuciones específicas: a) la formulación de un entorno de navegación sobre topografía real con observación local y origen aleatorio que inhabilita las soluciones tabulares; b) una comparación sistemática de paradigmas basados en valor y política frente a la frontera óptima de Dijkstra; y c) la documentación empírica de dos casos de estudio con resultados de aprendizaje negativos: la inestabilidad por *shaping* potencial y la divergencia de la función de valor con objetivos variables.

III. MARCO TEÓRICO

III-A. MDP y Q -learning

Un MDP se formaliza mediante una tupla constituida por un conjunto de estados s , un conjunto de acciones a , una función de transición de probabilidad $P(s'|s, a)$ y una función de recompensa $r(s, a)$. El objetivo fundamental es determinar una política π que maximice el retorno esperado descontado $\sum_t \gamma^t r_t$. El algoritmo Q -learning aproxima de forma no paramétrica la función de valor de la acción $Q(s, a)$ mediante la regla de actualización por diferencia temporal (Temporal Difference, TD):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]. \quad (1)$$

La política codiciosa (*greedy*) asociada se define como $\pi(s) = \arg \max_a Q(s, a)$.

III-B. Deep Q -Network (DQN)

Ante espacios de estados de alta dimensionalidad o continuos, la representación tabular resulta computacionalmente inviable. DQN [2] soluciona esta limitación parametrizando la función de valor mediante una red neuronal $Q(s, a; \theta)$ y minimizando la función de pérdida cuadrática:

$$\mathcal{L}(\theta) = (r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2, \quad (2)$$

empleando optimización por descenso de gradiente. Con el fin de mitigar la inestabilidad inherente al entrenamiento conjunto de aproximadores no lineales con aprendizaje por *bootstrapping*, DQN incorpora dos mecanismos de estabilización:

- **Reproducción de Experiencias** (*experience replay*): almacena las transiciones históricas en un búfer circular y extrae mini-lotes aleatorios para el entrenamiento, rompiendo la autocorrelación temporal entre muestras sucesivas.
- **Red Objetivo** (parámetros θ^-): una copia de la red neuronal con parámetros congelados calcula el valor del objetivo de Bellman, actualizándose de forma periódica. Esto evita la inestabilidad derivada de optimizar hacia un objetivo dinámico (*moving target*).

III-C. Double DQN

El operador de maximización empleado en el cálculo del objetivo de Bellman introduce un sesgo optimista sistemático debido a la acumulación de ruido positivo. Double DQN [3] soluciona esta sobreestimación desacoplando el proceso de selección de la acción del proceso de evaluación cuantitativa del estado resultante:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-), \quad (3)$$

donde los pesos activos θ seleccionan la acción óptima y los pesos congelados θ^- determinan su valor.

III-D. REINFORCE (gradiente de política)

A diferencia de los enfoques basados en valor, REINFORCE [6] parametriza directamente una política estocástica $\pi(a|s; \theta)$ y optimiza sus parámetros en la dirección del gradiente del retorno esperado:

$$\nabla_{\theta} J \approx \sum_t \nabla_{\theta} \log \pi(a_t|s_t; \theta) G_t, \quad (4)$$

donde G_t representa el retorno acumulado descontado obtenido a partir del paso temporal t . Para mitigar la alta varianza intrínseca a este estimador, se introduce una línea base mediante la normalización de los retornos G_t dentro de cada episodio, sustrayendo la media muestral y dividiendo por la desviación estándar del lote [8].

III-E. Reward shaping

En entornos con recompensas dispersas, el diseño de funciones de recompensa densas complementarias (*reward shaping*) acelera el proceso de aprendizaje. El rediseño potencial [5] formula una función de incentivo de la forma $F = \gamma \Phi(s') - \Phi(s)$, demostrando teóricamente la preservación de la política óptima. Sin embargo, en escenarios prácticos caracterizados por trayectorias extensas y factores de descuento $\gamma < 1$, dicha formulación introduce efectos colaterales indeseados debido a la acumulación del residuo temporal del descuento (Sec. VII).

IV. METODOLOGÍA

IV-A. El terreno y el MDP

Para el modelado del entorno se utiliza un recorte de aproximadamente 3×3 km correspondiente al Modelo Digital de Elevación (Digital Elevation Model, DEM) Copernicus GLO-30, obtenido a través del repositorio público AWS Open Data. La Fig. 1 ilustra la distribución espacial de la elevación y de la pendiente. El problema de navegación se formula mediante un MDP definido por:

- **Estados:** La posición discreta del agente en una cuadrícula de 108×108 celdas.
- **Acciones:** El conjunto finito de movimientos cardinales $\mathcal{A} = \{\text{arriba, abajo, izquierda, derecha}\}$.
- **Transición:** Determinística en el caso base, o estocástica según el modelo de perturbación por deslizamiento descrito en la Sec. VI-D.

- **Recompensa:** Una penalización por paso proporcional a la pendiente local, una penalización por colisión contra obstáculos inaccesibles, y una recompensa positiva terminal al alcanzar el objetivo.

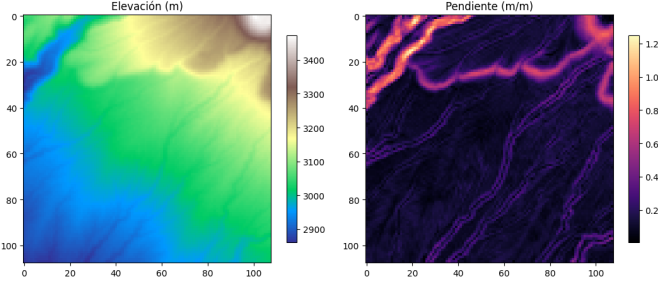


Figura 1. Terreno real (faldas del Misti): elevación (izq.) y pendiente (der.), malla 108×108 de 30 m.

IV-B. Observación local

El agente opera bajo condiciones de observación parcial del entorno. En cada paso temporal, recibe un parche local de 9×9 celdas de pendiente normalizada centrado en su posición (81 dimensiones), al cual se concatena el vector bidimensional de distancia relativa ($\Delta fila, \Delta columna$) hacia el objetivo, totalizando un vector de características de entrada de 83 dimensiones (Fig. 2). Esta representación es independiente de la escala global del mapa. Asimismo, la invarianza espacial de las observaciones locales facilita la capacidad de generalización del aproximador neuronal, a diferencia del esquema tabular clásico, cuya dimensionalidad escalaría a un orden impracticable de 10^8 combinaciones de pares de posición y objetivo.

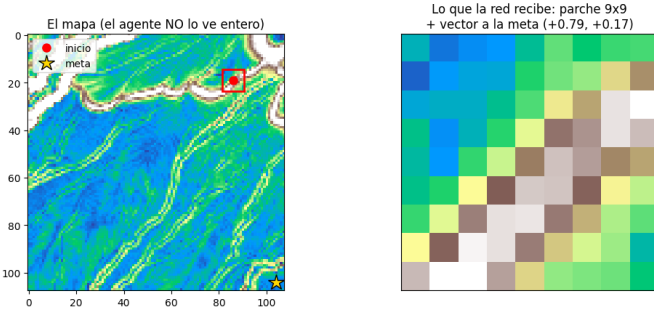


Figura 2. Observación local: el mapa completo con la ventana del agente (izq.) y el parche 9×9 que recibe la red (der.), más el vector a la meta.

IV-C. Meta fija con inicio aleatorio

El objetivo se establece en una coordenada espacial fija durante todo el proceso experimental, mientras que el estado inicial se muestrea aleatoriamente al inicio de cada episodio. Mantener el objetivo fijo estabiliza el entrenamiento al generar un campo de valores continuo y convergente. Por su parte, la variabilidad del origen previene la memorización de trayectorias únicas y obliga al agente a aprender una política

generalizable de aproximación al objetivo desde cualquier cuadrante del mapa.

IV-D. Reward shaping de progreso

Debido a la baja probabilidad de alcanzar de forma aleatoria una meta lejana en mapas de gran escala (recompensa dispersa), se implementa una señal de recompensa densa complementaria. La aplicación inicial de un rediseño potencial de la forma $\Phi(s) = -k \cdot d(s, meta)$ resultó en convergencia subóptima debido a la interacción con el descuento temporal (Sec. VII). En su lugar, se propone un **rediseño de recompensa por progreso**, formulado como $r += k [d(s, meta) - d(s', meta)]$. Esta formulación garantiza una acumulación neta de cero a lo largo de cualquier trayectoria cíclica cerrada, evitando los incentivos espurios del esquema potencial.

IV-E. Arquitectura e hiperparámetros

La arquitectura aproximadora empleada (tanto para la función de valor como para la política) consiste en un perceptrón multicapa estructurado como $83 \rightarrow 128 \rightarrow 128 \rightarrow 4$ con funciones de activación ReLU, sumando aproximadamente 27 mil parámetros optimizables. Se selecciona un factor de descuento $\gamma = 0,99$, una tasa de aprendizaje $\alpha = 10^{-3}$ utilizando el optimizador Adam, lotes de entrenamiento de 64 transiciones y un búfer de memoria de reproducción con capacidad para 10^5 registros. El entrenamiento emplea la función de pérdida de Huber y recorte de gradientes a una norma máxima de 10. La red objetivo θ^- se actualiza periódicamente cada 1000 iteraciones de optimización. La exploración se modela mediante una política ϵ -greedy con un decaimiento lineal de ϵ de 1,0 a 0,05 distribuido sobre los primeros $1,5 \times 10^5$ pasos. Cada experimento consta de 1500 episodios con un límite máximo de 500 pasos por episodio. Adicionalmente, se introduce un esquema curricular de entrenamiento en el cual los estados iniciales se restringen a un radio inicial de 20 celdas respecto a la meta, expandiéndose progresivamente hasta cubrir la totalidad del mapa durante la primera mitad del entrenamiento. Se fija una semilla pseudoaleatoria global en todas las simulaciones para garantizar la reproducibilidad. Los hiperparámetros completos se detallan en la Tabla II de la Sec. V.

V. IMPLEMENTACIÓN

V-A. Librerías y organización del código

El sistema de simulación y entrenamiento está desarrollado íntegramente en Python 3. Se emplea **PyTorch** para la definición y optimización de las redes neuronales; **NumPy** para la manipulación matricial del terreno, las máscaras de obstáculos y la generación de números pseudoaleatorios; **rasterio** para la lectura de datos del modelo digital de elevación en formato GeoTIFF y su recorte geográfico; y **Matplotlib** para la generación de gráficos. El algoritmo de Dijkstra se implementó de forma directa utilizando la estructura de cola de prioridad `heapq` de la biblioteca estándar de Python, reduciendo de este modo las dependencias externas del proyecto. No se utilizaron

Tabla I
ORGANIZACIÓN DEL CÓDIGO

Archivo	Contenido
entorno_drl.py	Formulación del MDP (terreno, observación, recompensa)
dqn.py	Arquitectura de red Q , búfer de reproducción y agente DQN
train.py	Rutinas de entrenamiento y validación de DQN
reinforce.py	Arquitectura de política y agente REINFORCE
train_pg.py	Rutina de optimización episódica para gradiente de política
baselines.py	Implementación de Dijkstra y cálculo de brechas de costo
datos.py	Adquisición y recorte geográfico del DEM
viz.py	Generación de gráficos y visualizaciones
01_meta_fija	Experimento principal DQN, trayectorias y análisis de ablación
02_estocastico	Simulación con transiciones estocásticas (deslizamiento)
03_double_dqn	Evaluación de Double DQN y análisis con metas aleatorias
04_reinforce	Comparación cuantitativa entre REINFORCE y DQN

librerías preexistentes de RL (como Gymnasium o Stable-Baselines); el diseño del entorno, el búfer de reproducción, los agentes y el ciclo de optimización se desarrollaron a medida, permitiendo la desactivación selectiva de componentes para los análisis de ablación.

El software se encuentra estructurado en módulos independientes complementados con cuadernos de trabajo (*notebooks*) específicos para cada escenario experimental (Tabla I). Esta modularización garantiza que todos los experimentos compartan de forma exacta la misma definición del entorno físico y las mismas rutinas de evaluación.

V-B. Procedimiento paso a paso

El flujo de procesamiento, desde la adquisición de datos primarios hasta la obtención de métricas de rendimiento, consta de seis etapas secuenciales:

1. **Adquisición de datos.** Se descarga el modelo digital de elevación Copernicus GLO-30 desde AWS Open Data y se realiza el recorte geográfico delimitado por las coordenadas $(-71,475, -16,375)$ y $(-71,445, -16,345)$, correspondiente a un área aproximada de 3×3 km. El recorte resultante se discretiza en una malla de 108×108 celdas con resolución espacial de 30 m.
2. **Derivación topográfica.** Se computa la pendiente mediante gradiente numérico a partir de la elevación. La pendiente obtenida determina el costo de tránsito por celda, definido como $c = 1 + 8 \|\nabla z\|$, y la máscara de obstáculos ($\|\nabla z\| > 0,60$). Se aplica un algoritmo de búsqueda en anchura (BFS) para extraer la componente conexa transitable de mayor dimensión, garantizando la existencia de trayectorias viables para cualquier par de inicio y meta seleccionado.

Algoritmo 1 Entrenamiento DQN con aprendizaje curricular

```

1: Inicializar red neuronal de valor  $Q_\theta$ ; copiar parámetros  $\theta^- \leftarrow \theta$ 
2: Inicializar búfer de reproducción  $\mathcal{D}$  con capacidad  $10^5$ ; paso global  $t \leftarrow 0$ 
3: for episodio  $e = 1$  hasta  $E$  do
4:    $d_{\max} \leftarrow 20 + \min(1, 2e/E) (216 - 20)$  ▷ Esquema curricular
5:    $s \leftarrow \text{RESET}(\text{estado inicial aleatorio con distancia } d \leq d_{\max})$ 
6:   repeat
7:      $\varepsilon \leftarrow \max(0, 0.5, 1 - t/150000)$ 
8:     Seleccionar  $a \leftarrow \arg \max_{a'} Q_\theta(s, a')$  con probabilidad  $1 - \varepsilon$ ; si no, acción aleatoria
9:     Ejecutar paso de simulación  $(s', r, \text{fin}) \leftarrow \text{STEP}(a)$ 
10:    Almacenar transición  $(s, a, r, s', \text{fin})$  en  $\mathcal{D}$ 
11:    if  $|\mathcal{D}| \geq 2000$  then
12:      Muestrear lote aleatorio de 64 transiciones de  $\mathcal{D}$ 
13:      Calcular objetivo  $y \leftarrow r + \gamma (1 - \text{fin}) \max_{a'} Q_{\theta^-}(s', a')$ 
14:      Actualizar  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{Huber}}(Q_\theta(s, a), y)$ 
15:      Recortar gradientes  $\|\nabla_\theta\| \leq 10$ 
16:      Cada 1000 iteraciones de optimización: copiar  $\theta^- \leftarrow \theta$ 
17:    end if
18:     $s \leftarrow s'$ ;  $t \leftarrow t + 1$ 
19:  until fin o límite de 500 pasos
20:  Cada 100 episodios: evaluar política greedy ( $\varepsilon = 0$ ) sobre pares de prueba fijos
21: end for

```

3. **Construcción de la observación.** En cada paso temporal, se extrae un parche local de 9×9 celdas de pendiente normalizada centrado en el agente, aplicando un relleno con valor 1,0 en los límites del mapa para simular paredes inaccesibles. A este vector se le concatena la posición relativa respecto a la meta normalizada por la diagonal del mapa, generando un vector de entrada de 83 dimensiones en el rango $[-1, 1]$.
4. **Optimización.** Se ejecuta el ciclo de entrenamiento descrito en el Algoritmo 1 para los métodos basados en valor. Para el algoritmo REINFORCE, se recopilan las transiciones del episodio completo antes de realizar una única actualización de los parámetros de la política.
5. **Evaluación periódica.** Cada 100 episodios (200 en las ablaciones y en el análisis de divergencia), se evalúa el desempeño de la política codiciosa ($\varepsilon = 0$) sobre un conjunto predefinido de 20 pares de prueba.
6. **Análisis de optimalidad.** Para las trayectorias exitosas en la fase de evaluación, se calcula el camino de costo mínimo mediante el algoritmo de Dijkstra global y se cuantifica la brecha relativa de costo mediante la métrica $100 (c_{\text{agente}}/c_{\text{Dijkstra}} - 1)$.

Las configuraciones de ablación se implementan mediante modificaciones unitarias al flujo del Algoritmo 1. Sin memoria de reproducción (*replay*), se omiten los pasos de almacenamiento y muestreo del búfer, optimizando directamente con la transición actual. Sin red objetivo, el cálculo de los valores objetivo de Bellman se realiza utilizando los pesos activos θ en lugar de θ^- . Para la variante Double DQN, el objetivo se calcula de acuerdo con la ec. (3).

V-C. Hiperparámetros

La Tabla II detalla la parametrización completa utilizada en los experimentos. Los valores se mantienen constantes en todos los escenarios evaluados, a excepción del número de episodios y la activación de los componentes de ablación.

Tabla II
HIPERPARÁMETROS DEL ENTORNO Y DE LOS AGENTES

Parámetro	Valor
<i>Entorno</i>	
Malla discreta / Dimensión de celda	108 × 108 / 30 m
Factor de escala de pendiente	8,0
Umbral de transitabilidad (obstáculo)	0,60
Dimensión del parche de observación	9 (vector de entrada de 83)
Límite de pasos por episodio	500
Distancia mínima de inicio	15 celdas
Penalización por paso	$-c/10$
Penalización por colisión / Meta	$-5 / +10$
Coefficiente de recompensa k	0,2
Probabilidad de deslizamiento	0 (o 0,25)
<i>Arquitectura (red de valor y política)</i>	
Estructura de capas	83 → 128 → 128 → 4, activación ReLU
Parámetros optimizables	≈ 27 mil
<i>DQN</i>	
Factor de descuento γ	0,99
Tasa de aprendizaje α	10^{-3} (Adam)
Dimensión de lote	64
Capacidad del búfer de reproducción	10^5 transiciones
Periodo de calentamiento	2000 transiciones
Frecuencia de sincronización de θ^-	cada 1000 iteraciones
Pérdida / Recorte de gradiente	Huber / $\ \nabla\ \leq 10$
Perfil de exploración ε	Decaimiento lineal de 1,0 a 0,05
Horizonte de decaimiento	$1,5 \times 10^5$ pasos
<i>REINFORCE</i>	
Parámetros γ / α	$0,99 / 10^{-3}$ (Adam)
Línea base	Normalización de retornos G_t
Frecuencia de actualización	Una por episodio
<i>Común</i>	
Episodios totales	1500 (1600 en análisis de divergencia)
Esquema curricular $d_{\max}$	Incremento lineal 20 → 216 (primera mitad)
Evaluación	20 pares fijos, $\varepsilon = 0$

V-D. Reproducibilidad

El análisis comparativo de las diversas configuraciones (DQN completo, sin búfer de reproducción, sin red objetivo y Double DQN) se realiza utilizando instancias idénticas del entorno físico y semillas globales unificadas, garantizando que el algoritmo activo constituya la única variable experimental. Con el fin de asegurar la reproducibilidad de las simulaciones, se fija la semilla pseudoaleatoria en 42 al inicializar NumPy, PyTorch y el generador de entornos. Los 20 pares fijos de evaluación se generan previamente empleando la semilla 12345; por construcción, estos estados iniciales no se presentan al agente durante las fases de entrenamiento.

Debido a la dimensión acotada del aproximador (aproximadamente 27 mil parámetros), el proceso de optimización

es ejecutable en CPU; no obstante, el código incluye soporte y detección automática para aceleración por GPU. El tiempo medio de entrenamiento para un ciclo de 1500 episodios es de aproximadamente 12 minutos. La generación de la totalidad de gráficos del artículo se realiza mediante la ejecución del script `gen_figuras_informe.py`. Por último, los datos del DEM no requieren versionamiento en el repositorio, puesto que el módulo `datos.py` automatiza su descarga desde el repositorio de Copernicus y aplica el recorte de coordenadas, garantizando la consistencia del terreno en cualquier plataforma de cómputo.

VI. EXPERIMENTOS Y RESULTADOS

VI-A. Configuración

El procedimiento de evaluación cuantifica dos métricas clave de la política codiciosa ($\varepsilon = 0$): la tasa de éxito en alcanzar la meta y la longitud media de la trayectoria (número de pasos). La evaluación se efectúa sobre 20 configuraciones de inicio fijas generadas de forma independiente a la muestra de entrenamiento (es decir, estados no observados previamente por el agente). Asimismo, se evalúa la desviación de costo acumulado respecto al camino óptimo global determinado mediante el algoritmo de Dijkstra.

VI-B. DQN con meta fija

La evolución del aprendizaje de la variante DQN principal se ilustra en la Fig. 3. El agente converge de manera estable a una **tasa de éxito del 100 %** (20 de 20 trayectorias de prueba resueltas). La Fig. 4 presenta una comparación espacial de las trayectorias aprendidas por el agente en contraste con las trayectorias óptimas de Dijkstra. Evaluado sobre el conjunto completo de prueba, el agente presenta una brecha media de costo de únicamente el **3,3 %**, un valor notable considerando que la red neuronal opera bajo observación parcial y no posee conocimiento del mapa topográfico global.

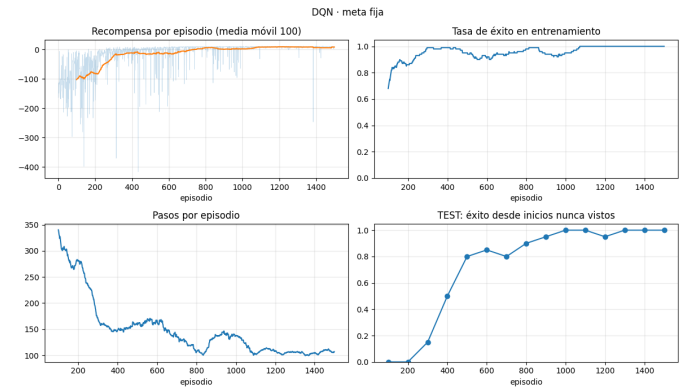


Figura 3. Curva de aprendizaje de DQN con meta fija: recompensa, tasa de éxito de entrenamiento, pasos y test sobre inicios no vistos.

VI-C. Ablaciones

Se realiza un estudio de ablación análogo al planteado por Liu y Zou [4] desactivando de forma individual los mecanismos de estabilización del algoritmo DQN (Tabla III, Fig. 5).

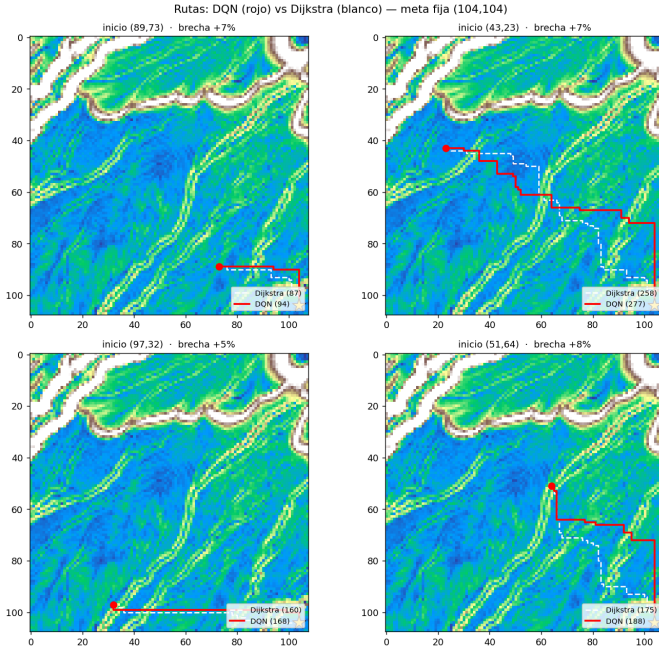


Figura 4. Rutas aprendidas por DQN (rojo) frente al óptimo exacto de Dijkstra (blanco), para cuatro pares inicio–meta de test.

El impacto experimental de ambas exclusiones es crítico: la desactivación de la red objetivo (θ^-) impide la convergencia del aprendizaje (0 % de éxito), comportamiento similar al obtenido al omitir el búfer de reproducción de experiencias (0 % de éxito). Estos resultados evidencian la complementariedad e indispensabilidad de ambos componentes. La variante Double DQN presenta un rendimiento equivalente a la versión DQN estándar (100 % de éxito).

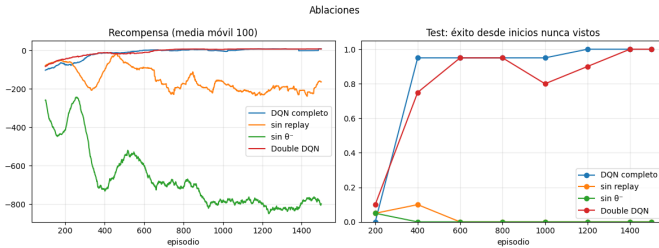


Figura 5. Ablaciones: DQN completo, sin *replay*, sin red objetivo (θ^-) y Double DQN.

Tabla III
ABLACIONES: TASA DE ÉXITO EN EL TEST

Configuración	Éxito
DQN completo	100 %
sin <i>replay</i>	0 %
sin red objetivo (θ^-)	0 %
Double DQN	100 %

VI-D. Transición estocástica

Se evalúa la robustez del agente frente a perturbaciones del entorno mediante la introducción de transiciones estocásticas (deslizamiento). Específicamente, se modela una probabilidad de deslizamiento proporcional a la pendiente local, bajo la cual la acción seleccionada es reemplazada por una dirección cardinal aleatoria. La Fig. 6 compara el desempeño del agente en entornos deterministas frente a estocásticos. En ambas condiciones se alcanza el 100 % de éxito; no obstante, el entorno estocástico requiere una longitud media de trayectoria ligeramente mayor (102 pasos frente a 96). Estos resultados demuestran la adaptabilidad de la política ante incertidumbre de transición, una propiedad que la formulación clásica de Dijkstra no aborda.

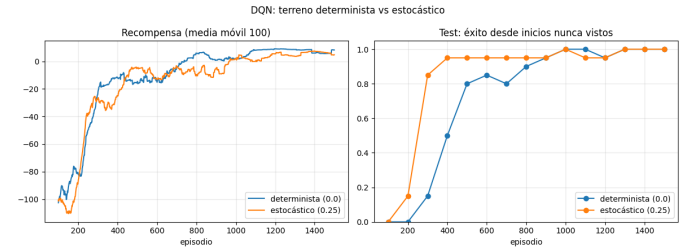


Figura 6. DQN en terreno determinista vs estocástico (resbalón 0,25).

VI-E. Valor vs política: DQN y REINFORCE

La comparación de desempeño entre el enfoque basado en valor (DQN) y el basado en política (REINFORCE) se presenta en la Fig. 7. Ambos paradigmas alcanzan tasas de éxito elevadas (DQN: 100 %, REINFORCE: 95 %). Sin embargo, la dinámica de aprendizaje presenta diferencias sustanciales: DQN converge de forma suave y estable debido al efecto del búfer de reproducción y la red objetivo, mientras que REINFORCE exhibe una varianza significativa, con oscilaciones marcadas en el retorno episódico. Esta divergencia ilustra las características clásicas asociadas a cada familia de algoritmos.

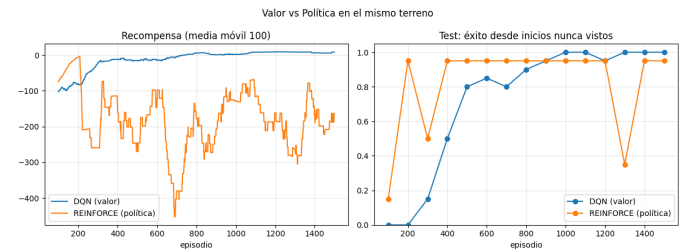


Figura 7. Valor vs política: DQN y REINFORCE en el mismo terreno.

VI-F. Divergencia con meta aleatoria

La Fig. 8 presenta el rendimiento de DQN ante la introducción de objetivos variables en cada episodio (configuración *goal-conditioned*). Se observa que la magnitud media de los valores Q se mantiene estable hasta el episodio 1000, momento

a partir del cual experimenta una divergencia asintótica, escalando de un valor de ~ 1 a más de 2400. De forma simultánea, la tasa de éxito colapsa por debajo del 10 %. Esta inestabilidad se atribuye al sesgo de sobreestimación del operador de maximización acumulado mediante *bootstrapping*, justificando la adopción de un esquema de meta fija en el presente estudio.

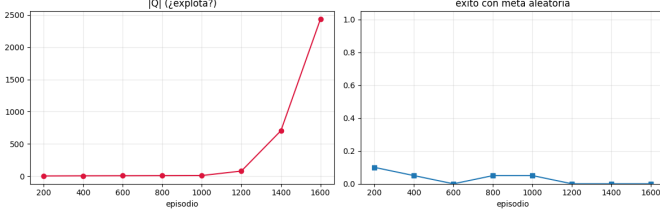


Figura 8. Con meta aleatoria, la magnitud media de Q (izq.) crece sin control (divergencia) y el éxito (der.) no se sostiene.

VII. DISCUSIÓN

VII-A. La trampa del shaping potencial

El rediseño de recompensa (*reward shaping*) potencial formulado bajo un factor de descuento temporal $\gamma < 1$ induce un sesgo residual positivo proporcional a la distancia al objetivo. Específicamente, al evaluar una trayectoria cíclica cerrada entre dos celdas adyacentes X e Y distantes de la meta, la suma de las recompensas potenciales intermedias no se anula, sino que colapsa (*telescoping*) a un valor neto de $k(1 - \gamma)(d_X + d_Y) > 0$. Para distancias al objetivo elevadas, este residuo supera el costo de paso negativo del entorno, resultando en un incentivo neto positivo por oscilar indefinidamente entre celdas lejanas (efecto *ping-pong*). En consecuencia, la política codiciosa converge hacia bucles infinitos locales. El esquema de *shaping* de progreso propuesto elimina este residuo sistemático, resolviendo la anomalía. Este fenómeno ilustra cómo una propiedad teóricamente robusta (la invariancia de la política óptima en entornos sin descuento o con trayectorias finitas sin descuento) puede degradarse en la práctica debido a su interacción con el descuento episódico.

VII-B. Divergencia y la elección de meta fija

Bajo entornos con objetivos variables, el sesgo de sobreestimación introducido por el operador máx se propaga y amplifica de forma recursiva a través del proceso de *bootstrapping*, induciendo la divergencia exponencial observada de los valores Q . Si bien el algoritmo Double DQN ralentiza este proceso de desestabilización, no logra evitar la divergencia en la configuración evaluada. Por el contrario, la fijación del objetivo genera un campo de valores único, estacionario y convergente. Dicha restricción metodológica no representa una limitación accidental del diseño, sino una decisión fundamentada orientada a garantizar la estabilidad numérica del aprendizaje.

VII-C. Limitación: política de meta única

La principal limitación de la política desarrollada radica en su especificidad respecto al objetivo: aunque el agente

demuestra una notable capacidad de generalización sobre los estados iniciales, la política está parametrizada para una meta estática específica. Modificar la ubicación del objetivo requiere reiniciar el proceso de entrenamiento. Extender la capacidad de generalización hacia metas arbitrarias requiere el empleo de arquitecturas condicionales y mecanismos de regularización robustos contra la divergencia, aspectos propuestos como líneas prioritarias de investigación futura.

VIII. CONCLUSIONES

En el presente estudio, se ha desarrollado e implementado de forma exitosa un marco experimental para la navegación autónoma de costo mínimo sobre topografía real, utilizando Aprendizaje por Refuerzo Profundo bajo condiciones de observación parcial y estados iniciales variables. A partir de los resultados cuantitativos y análisis empíricos presentados, se formulan las siguientes conclusiones:

- **Generalización y Optimalidad:** El agente entrenado demostró una notable capacidad de generalización al alcanzar la meta desde configuraciones de inicio no observadas durante el entrenamiento, registrando una tasa de éxito del 100 %. Las trayectorias generadas exhibieron una desviación media de costo de solo el 3,3 % en comparación con la frontera óptima global calculada por el algoritmo de Dijkstra.
- **Robustez ante Incertidumbre:** Bajo transiciones estocásticas que modelan condiciones de deslizamiento físico, la política aprendida exhibió resiliencia al mantener el 100 % de éxito, requiriendo un incremento de solo 6 pasos en promedio respecto a la simulación determinista (102 frente a 96 pasos). Esto evidencia la adaptabilidad del DRL frente a la rigidez de la planificación clásica.
- **Indispensabilidad de Mecanismos en DQN:** El análisis de ablación demostró que tanto la memoria de reproducción de experiencias como la red objetivo congelada (θ^-) son mecanismos críticos para la estabilidad numérica del aprendizaje. La desactivación individual de cualquiera de ellos provocó el colapso del entrenamiento, resultando en una tasa de éxito del 0 %.
- **Sensibilidad al Diseño de Recompensas:** Se caracterizó empíricamente que el rediseño potencial tradicional ($F = \gamma\Phi(s') - \Phi(s)$) induce comportamientos espurios (bucles infinitos locales) en trayectorias extensas cuando se opera con factores de descuento $\gamma < 1$. Por el contrario, el esquema de recompensa por progreso propuesto resolvió esta inestabilidad al eliminar los residuos acumulados en ciclos cerrados.
- **Inestabilidad ante Objetivos Variables:** Se demostró que la formulación estándar de DQN es propensa a la divergencia asintótica de los valores Q (escalando de ~ 1 a más de 2400) ante metas aleatorias, debido a la realimentación del sesgo del operador máx. Esto justifica de manera metodológica la adopción de una política parametrizada para un objetivo estacionario en el presente trabajo.

IX. TRABAJO FUTURO

Como líneas de trabajo futuro, se proponen: a) la extensión del algoritmo a políticas condicionadas al objetivo (*goal-conditioned*) incorporando técnicas de reetiquetado de experiencias (Hindsight Experience Replay, HER) o muestreo priorizado (PER) para estabilizar la función de valor; b) la escalabilidad del modelo a entornos geográficos de mayor dimensión; y c) la evaluación de arquitecturas de gradiente de política avanzadas de tipo actor-crítico.

APÉNDICE

El código fuente completo de este proyecto, incluyendo los algoritmos implementados, los entornos de simulación y las rutinas de visualización, se encuentra disponible de forma pública en el siguiente repositorio de GitHub:

<https://github.com/uni-maestria/reinforcement-learning-DEM>

REFERENCIAS

- [1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [2] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
- [3] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *AAAI*, 2016.
- [4] R. Liu and J. Zou, "The effects of memory replay in reinforcement learning," in *Allerton Conf.*, IEEE, 2018.
- [5] A. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: theory and application to reward shaping," in *ICML*, 1999.
- [6] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992.
- [7] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *ICML*, 2016.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [9] T. Schaul, D. Horgan, K. Gregor, and D. Silver, "Universal value function approximators," in *ICML*, 2015.
- [10] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *ICLR*, 2016.
- [11] M. Andrychowicz *et al.*, "Hindsight experience replay," in *NeurIPS*, 2017.